
Documento de Visión y Alcance

Proyecto: Bot de atención por WhatsApp para gestión de tickets de soporte **Organización:** Axión Lift **Área inicial:** Sistema 4.0 **Autor:** Tadeo Cervellati **Tutor:** Fernando Vatri **Versión:** 0.1 (borrador) **Fecha:** 06/2026

1. Introducción

1.1 Propósito del documento

Este documento define la visión general, los objetivos y el alcance del sistema a desarrollar. Establece una base común de entendimiento entre el equipo de desarrollo y los interesados, y sirve como referencia para la validación del alcance antes de avanzar a las fases de diseño y desarrollo.

1.2 Alcance del documento

Cubre la descripción del problema, la solución propuesta, los interesados, las características principales del producto y las restricciones del proyecto. No incluye el detalle de requerimientos funcionales ni el diseño técnico, que se documentan en entregables posteriores.

2. Posicionamiento

2.1 Descripción del problema

Actualmente, los empleados que necesitan soporte deben cargar un ticket a través del formulario estándar de GLPI. Este proceso puede resultar burocrático y poco intuitivo para el usuario promedio (alrededor de 30 años, familiarizado con la mensajería instantánea), ~~lo que genera fricción al momento de reportar incidentes y puede derivar en reportes incompletos o en su omisión.~~

2.2 Sentencia de posición del producto

Elemento	Descripción
Para	los empleados de las distintas áreas de la empresa (planta, ventas, etc.) que necesitan reportar un incidente al soporte.

Quienes	encuentran el formulario de GLPI poco amigable y burocrático.
El producto es	un bot de atención conversacional por WhatsApp.
Que	permite describir el problema en lenguaje natural y crea el ticket en GLPI de forma automática, derivando al área Sistema 4.0.
A diferencia de	el formulario web tradicional de GLPI.
Nuestro producto	ofrece una experiencia más humana y cercana, sin perder la seriedad ni la trazabilidad del proceso.

3. Objetivos

3.1 Objetivo general

Desarrollar un bot conversacional que sirva como alternativa al formulario de GLPI de IT y Sistemas - Solicitudes, permitiendo que el usuario describa su problema por chat para que el sistema clasifique el incidente y genere automáticamente el ticket de soporte.

3.2 Objetivos específicos

- Recibir mensajes de los usuarios a través de WhatsApp (Meta Cloud API).
- Clasificar el incidente mediante IA a partir de texto libre, sin menús rígidos.
- Consultar dinámicamente las categorías desde GLPI para no acoplar el bot a una configuración fija.
- Crear el ticket en GLPI a través de su API REST.
- Identificar al usuario que escribe, vinculándolo con el teléfono asociado a GLPI o solicitando sus datos cuando no haya coincidencia.
- Dejar documentación técnica clara siguiendo el Proceso Unificado.

4. Interesados (Stakeholders)

Interesado	Rol	Interés / Influencia
Tadeo Cervellati	Desarrollador / Analista	Ejecuta el proyecto.
Fernando	Tutor técnico	Guía técnica y validación. Influencia alta.

Cristian	Gerencia	Aprobación de factibilidad y costos. Influencia alta.
Área Sistema 4.0	Operadores de GLPI	Reciben y resuelven los tickets que genera el bot. Dueños de la cola de soporte. Influencia alta.
Usuarios finales	Empleados de otras áreas (planta, ventas, etc.)	Reportan incidentes a través del bot.

5. Vista general del producto

5.1 Perspectiva del producto

El bot actúa como un conector (patrón *bridge* / ETL) entre dos sistemas: WhatsApp como canal de entrada y GLPI como sistema de tickets. No reemplaza a GLPI ni accede directamente a su base de datos MariaDB; se comunica exclusivamente a través de la API REST de GLPI, priorizando consultas (GET) y usando POST únicamente para crear tickets.

El flujo de tráfico previsto es:

Meta (WhatsApp) → Cloudflare (DNS/proxy) → Nginx (reverse proxy) → port forwarding de Docker → contenedor del bot → API REST de GLPI

5.2 Características principales

- Recepción de mensajes vía webhook de WhatsApp.
- Clasificación de incidentes mediante IA (Gemini como motor principal; OpenAI disponible para pruebas comparativas).
- Consulta dinámica de categorías de GLPI.
- Creación automática de tickets.
- Identificación del usuario que reporta.
- Diseño modular para extender el bot a otras áreas a futuro.

6. Alcance

6.1 Alcance del producto (qué hace el sistema)

- Recibir, vía WhatsApp, reportes de incidentes provenientes de empleados de cualquier área de la empresa.
- Clasificar el incidente y crear el ticket en GLPI automáticamente, derivando al área de soporte Sistema 4.0.
- Identificar al usuario que escribe.

6.2 Fuera de alcance (qué NO hace, por ahora)

- Atención a áreas distintas de Sistema 4.0 (queda como extensión futura).
- Resolución automática del incidente (el bot crea el ticket, no lo soluciona).
- Acceso o escritura directa sobre la base de datos MariaDB de GLPI.
- Pipelines de deploy automático avanzados (CI/CD); se prioriza un despliegue simple y mantenible.
- Funcionalidad de cotización para ventas externas (idea a futuro, no en esta etapa).

6.3 Escalabilidad prevista

El diseño debe ser modular y reutilizable para habilitar, en etapas posteriores, su aplicación a otras áreas (mantenimiento, diseño) o casos de uso (por ejemplo, un bot de cotización).

7. Restricciones

Tipo	Restricción
Tiempo	Plazo de 2 meses (proyecto de facultad).
Metodología	Debe seguir el Proceso Unificado (análisis, diseño, desarrollo) con documentación técnica obligatoria.
Técnica	El bot se conecta a GLPI solo por API REST, no por base de datos. Las categorías se consultan en tiempo real, no se hardcodean.
Infraestructura	Despliegue en contenedores Docker gestionados con Portainer Community Edition, desde repo de GitHub vía stack / Docker Compose.
Económica	A definir con gerencia: costos de WhatsApp API y uso de IA. [Pendiente de relevamiento.]

8. Riesgos y cuestiones abiertas

- **Identificación del usuario:** definir cómo vincular el número de teléfono con el usuario del Active Directory / LDAP, o el flujo alternativo de pedir nombre y área cuando no haya coincidencia. (*Desafío abierto a resolver en análisis.*)
- **Costos de WhatsApp API:** relevar el modelo de precios de Meta Cloud API y negociar con gerencia.
- **Curva de aprendizaje de Docker/Portainer:** el equipo es principiante en contenedores; se prevé acompañamiento del área técnica de Axió Lift.
- **Canal definitivo:** WhatsApp es la opción priorizada, pero se mencionaron Google Chat y Telegram como alternativas.

La **escala de importancia** es: 1 = poco importante, 2 = moderadamente importante, 3 = importante, 4 = muy importante.

Requerimientos Funcionales

Identificador	Requisito Funcional	Importancia
01_REQ_FUN	El sistema debe recibir los mensajes de los usuarios a través de WhatsApp, mediante el webhook de la Meta Cloud API.	4
02_REQ_FUN	El sistema debe sostener un diálogo en lenguaje natural que permita al usuario describir su problema con texto libre, sin menús rígidos.	4
03_REQ_FUN	El sistema debe identificar al usuario que escribe, vinculando su número de teléfono con su cuenta en GLPI. Si el número no está registrado, el sistema no crea el ticket e informa al usuario que no está habilitado.	3
04_REQ_FUN	Cuando el número está registrado, el sistema debe confirmar la identidad del solicitante presentando el nombre asociado en GLPI para que el usuario lo confirme o corrija, en lugar de asumirlo a partir del número.	4
05_REQ_FUN	El sistema debe clasificar automáticamente el incidente a partir de la descripción del usuario, utilizando un modelo de IA.	4
06_REQ_FUN	El sistema debe consultar dinámicamente las categorías de ticket disponibles desde GLPI (vía API REST, GET), sin tenerlas hardcodeadas.	4
07_REQ_FUN	El sistema debe crear el ticket en GLPI mediante su API REST (POST), asignándole la categoría correspondiente y derivándolo al área de soporte Sistema 4.0.	4
08_REQ_FUN	El sistema debe confirmar al usuario, por WhatsApp, la creación exitosa del ticket e informar su número/identificador.	3

09_REQ_FUN	El sistema debe manejar los errores de comunicación con GLPI (no disponible, fallo en la creación) e informar adecuadamente al usuario.	3
10_REQ_FUN	El sistema debe registrar (log) las interacciones y operaciones realizadas, para trazabilidad y diagnóstico.	2
11_REQ_FUN	El sistema debe mantener el estado de cada conversación entre mensajes (contexto multi-turno), recordando en qué punto del flujo está cada usuario para retomar la interacción correctamente.	4

Requerimientos No Funcionales

Identificador	Categoría	Requisito No Funcional	Importancia
01_REQ_NF	Modularidad	El diseño debe ser modular y reutilizable, de modo que el bot pueda extenderse a otras áreas de soporte sin reescribir su núcleo.	4
02_REQ_NF	Integración	El sistema debe comunicarse con GLPI exclusivamente a través de su API REST, sin acceder directamente a la base de datos MariaDB.	4
03_REQ_NF	Seguridad	El sistema debe validar el origen de las peticiones del webhook (token de verificación de Meta) y resguardar de forma segura las credenciales de GLPI y las API keys de IA.	4
04_REQ_NF	Privacidad	Los datos de los usuarios obtenidos de GLPI deben tratarse de forma confidencial y usarse solo para asociar el ticket.	3
05_REQ_NF	Rendimiento	El bot debe responder al usuario en un tiempo razonable [definir umbral, ej. < 5 s en condiciones normales].	3
06_REQ_NF	Confiabilidad	El sistema debe evitar la pérdida de reportes ante fallos transitorios (ej. reintentos o manejo controlado de errores).	3

07_REQ_NF	Despliegue	El sistema debe ejecutarse en un contenedor Docker gestionado con Portainer, desplegado desde el repositorio de GitHub vía stack / Docker Compose.	4
08_REQ_NF	Mantenibilidad	El código y la configuración deben estar documentados siguiendo el Proceso Unificado, de forma clara y mantenible.	4
09_REQ_NF	Usabilidad	La interacción debe ser en español y con un tono cercano, manteniendo la formalidad del proceso de soporte.	3
10_REQ_NF	Persistencia	El estado de las conversaciones se almacenará en una base SQLite embebida, alojada en un volumen persistente del contenedor del bot, sin requerir un servicio de base de datos adicional.	

Épicas e Historias de Usuario

EP01 — Atención conversacional por WhatsApp

Agrupar: 01_REQ_FUN, 02_REQ_FUN, 08_REQ_FUN, 11_REQ_FUN

HU01 — Reportar un problema por chat Como empleado de la empresa, quiero escribirle al bot por WhatsApp y describir mi problema con mis propias palabras, para reportar un incidente sin tener que usar el formulario de GLPI.

Criterios de aceptación:

- El bot recibe el mensaje del usuario a través del webhook de WhatsApp.
- El usuario puede explicar el problema en texto libre, sin elegir opciones de un menú o como prefiera el usuario.
- El bot responde dentro del mismo hilo de conversación.

HU02 — Recibir confirmación del ticket Como empleado, quiero recibir una confirmación con el número de ticket una vez creado, para saber que mi reporte quedó registrado.

Criterios de aceptación:

- Al crearse el ticket, el bot envía un mensaje con su identificador.

- Si no se pudo crear, el bot informa el problema en lugar de quedarse en silencio y brinda una alternativa.

HU10 — Conversación de varios turnos Como empleado, quiero poder responderle al bot en varios mensajes y que no pierda el hilo de lo que veníamos hablando, para describir mi problema de forma natural sin tener que mandar todo de una.

Criterios de aceptación:

- El bot recuerda el contexto de la conversación entre un mensaje y el siguiente.
 - Si el usuario responde a una pregunta del bot, el bot continúa el flujo desde donde estaba.
 - El estado de la conversación sobrevive a un reinicio del contenedor.
-

EP02 — Identificación del solicitante

Agrupar: 03_REQ_FUN, 04_REQ_FUN

HU03 — Reconocer al usuario automáticamente Como operador de Sistema 4.0, quiero que el bot identifique automáticamente quién reporta vinculando su número de teléfono con el usuario de GLPI, para saber de qué persona y área proviene el ticket sin tener que preguntarlo.

Criterios de aceptación:

- El bot busca el número de teléfono en GLPI.
- Si hay coincidencia, asocia el nombre y el área al ticket de forma automática.
- Si el número no está registrado, el sistema no crea el ticket e informa al usuario que no está habilitado.

HU12 — Solo empleados registrados Como responsable de Sistema 4.0, quiero que solo los números registrados en GLPI puedan generar tickets, para que nadie externo con el número del bot sature la cola de soporte.

Criterios de aceptación:

- El bot verifica el número contra GLPI antes de crear cualquier ticket.
- Si el número no está registrado, no se crea el ticket y se le avisa al usuario.
- El intento queda registrado en el log.

HU13 — Confirmar quién soy Como empleado, quiero que el bot confirme conmigo quién soy en vez de asumirlo por el número, para que el ticket quede a nombre de la persona correcta aunque el celular haya cambiado de dueño.

Criterios de aceptación:

- Si el número está registrado, el bot presenta el nombre asociado en GLPI y pide confirmación.
 - Si el usuario confirma, se usa ese nombre.
 - Si el usuario corrige, el bot toma el nombre corregido.
-

EP03 — Clasificación y registro del incidente en GLPI

Agrupar: 05_REQ_FUN, 06_REQ_FUN, 07_REQ_FUN

HU05 — Clasificación automática del incidente Como empleado, quiero que el bot entienda y clasifique mi problema automáticamente, para no tener que elegir la categoría yo mismo de una lista.

Criterios de aceptación:

- El bot determina la categoría del incidente a partir de la descripción del usuario, usando IA.
- La categoría asignada corresponde a una de las categorías válidas existentes en GLPI.

HU06 — Categorías dinámicas desde GLPI Como administrador del sistema, quiero que las categorías se lean dinámicamente desde GLPI, para que el bot no se rompa si alguien las modifica.

Criterios de aceptación:

- El bot obtiene las categorías consultando la API REST de GLPI (GET), no de una lista hardcodeada.
- Si se agrega o cambia una categoría en GLPI, el bot la considera sin necesidad de tocar el código.

HU07 — Creación del ticket en GLPI Como operador de Sistema 4.0, quiero que el ticket se cree en GLPI con la categoría correcta y derivado a mi área, para gestionarlo desde mi flujo habitual.

Criterios de aceptación:

- El bot crea el ticket vía API REST (POST) con la categoría asignada.
 - El ticket queda derivado al área de soporte Sistema 4.0.
 - El ticket incluye la descripción original del usuario y sus datos de identificación.
-

EP04 — Operación confiable del servicio

Agrupar: 09_REQ_FUN, 10_REQ_FUN

HU08 — Aviso ante fallos Como empleado, quiero que el bot me avise si algo falla al crear el ticket, para saber que debo reintentar o usar otro canal.

Criterios de aceptación:

- Si GLPI no responde o devuelve error, el bot informa al usuario de forma clara.
- El error queda registrado para diagnóstico.

HU09 — Trazabilidad de las operaciones Como administrador/desarrollador, quiero que el sistema registre las interacciones y los errores, para diagnosticar problemas y auditar el funcionamiento.

Criterios de aceptación:

- Cada operación relevante (mensaje recibido, ticket creado, error) queda registrada en un log.
- Los registros permiten reconstruir qué pasó ante un reclamo o falla.

ID	Caso de uso	Actor
CU01	Reportar problema por WhatsApp	Usuario
CU02	Describir problema en lenguaje natural	Usuario
CU03	Confirmar o corregir datos (identificación)	Usuario
CU04	Recibir confirmación del ticket creado	Usuario
CU05	Clasificar problema con IA (Gemini) «include»	—
CU06	Consultar categorías dinámicas de GLPI «include»	—
CU07	Crear ticket en GLPI vía API «include»	—
CU08	Mantener estado de la conversación «include»	—

CU09	Ver ticket en GLPI con datos estructurados	Sistema 4.0
CU10	Asignar prioridad y técnico al ticket	Sistema 4.0
CU11	Resolver y cerrar el ticket	Sistema 4.0

Matriz de Trazabilidad — Requerimientos Funcionales

Requerimiento	Épica	Historia de Usuario	Caso de Uso
01_REQ_FUN	EP01	HU01	CU01
02_REQ_FUN	EP01	HU01	CU02
03_REQ_FUN	EP02	HU03	CU03
04_REQ_FUN	EP02	HU04	CU03
05_REQ_FUN	EP03	HU05	CU05
06_REQ_FUN	EP03	HU06	CU06
07_REQ_FUN	EP03	HU07	CU07
08_REQ_FUN	EP01	HU02	CU04
09_REQ_FUN	EP04	HU08	CU07 / CU04 (manejo de su error)
10_REQ_FUN	EP04	HU09	Transversal (sin CU propio)
11_REQ_FUN	EP01	HU10	CU08

Matriz de Trazabilidad — Requerimientos No Funcionales

Requerimiento	Categoría	Alcance / RF o Épicas afectadas
01_REQ_NF	Modularidad	Todo el diseño (todas las épicas)
02_REQ_NF	Integración	EP03 (06, 07_REQ_FUN)
03_REQ_NF	Seguridad	EP01 (01), EP02 (03), EP03 (06, 07_REQ_FUN)

04_REQ_NF	Privacidad	EP02 (03, 04_REQ_FUN)
05_REQ_NF	Rendimiento	EP01 y EP03 (todo el ciclo de respuesta)
06_REQ_NF	Confiabilidad	EP04 (09), EP03 (07_REQ_FUN)
07_REQ_NF	Despliegue	Todo el sistema
08_REQ_NF	Mantenibilidad	Todo el sistema
09_REQ_NF	Usabilidad	EP01 (02, 08_REQ_FUN)
10_REQ_NF	Persistencia	EP01 (11_REQ_FUN)

Arquitectura e Infraestructura

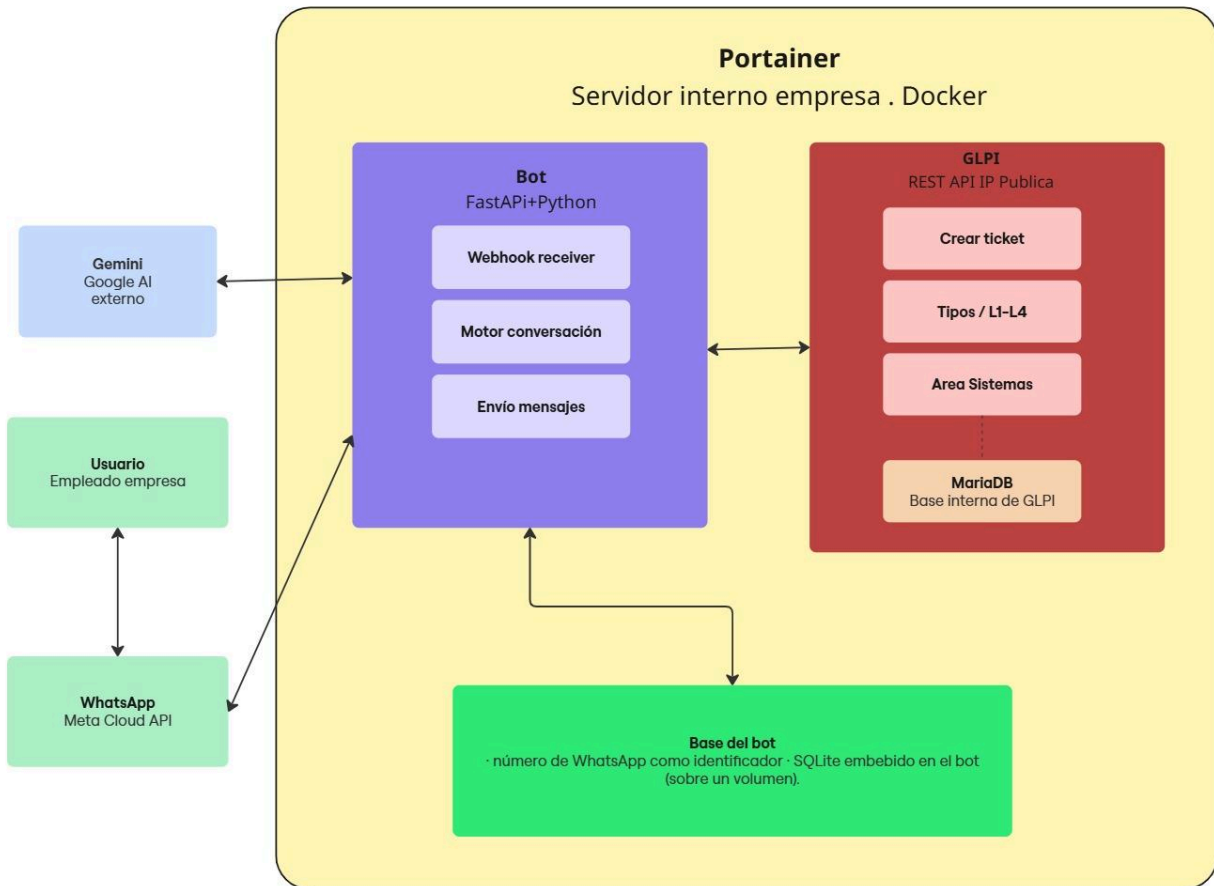
Introducción

Esta sección describe la arquitectura y la infraestructura sobre la que funcionará el bot. Cubre tres vistas complementarias: la arquitectura de componentes (qué piezas forman el sistema y cómo se relacionan), la estructura por capas (dónde "vive" físicamente el bot), y el camino que recorre un mensaje desde el celular del usuario hasta el contenedor del bot. Buena parte de la infraestructura descrita ya existe en la empresa; la responsabilidad del proyecto es construir el bot e integrarlo en ese entorno.

1. Arquitectura de componentes

El sistema se compone del bot, los servicios externos con los que se comunica y la base de datos propia del bot. Todo lo interno corre sobre el servidor de la empresa, gestionado con Docker y Portainer.

[Figura 1: Arquitectura de componentes]



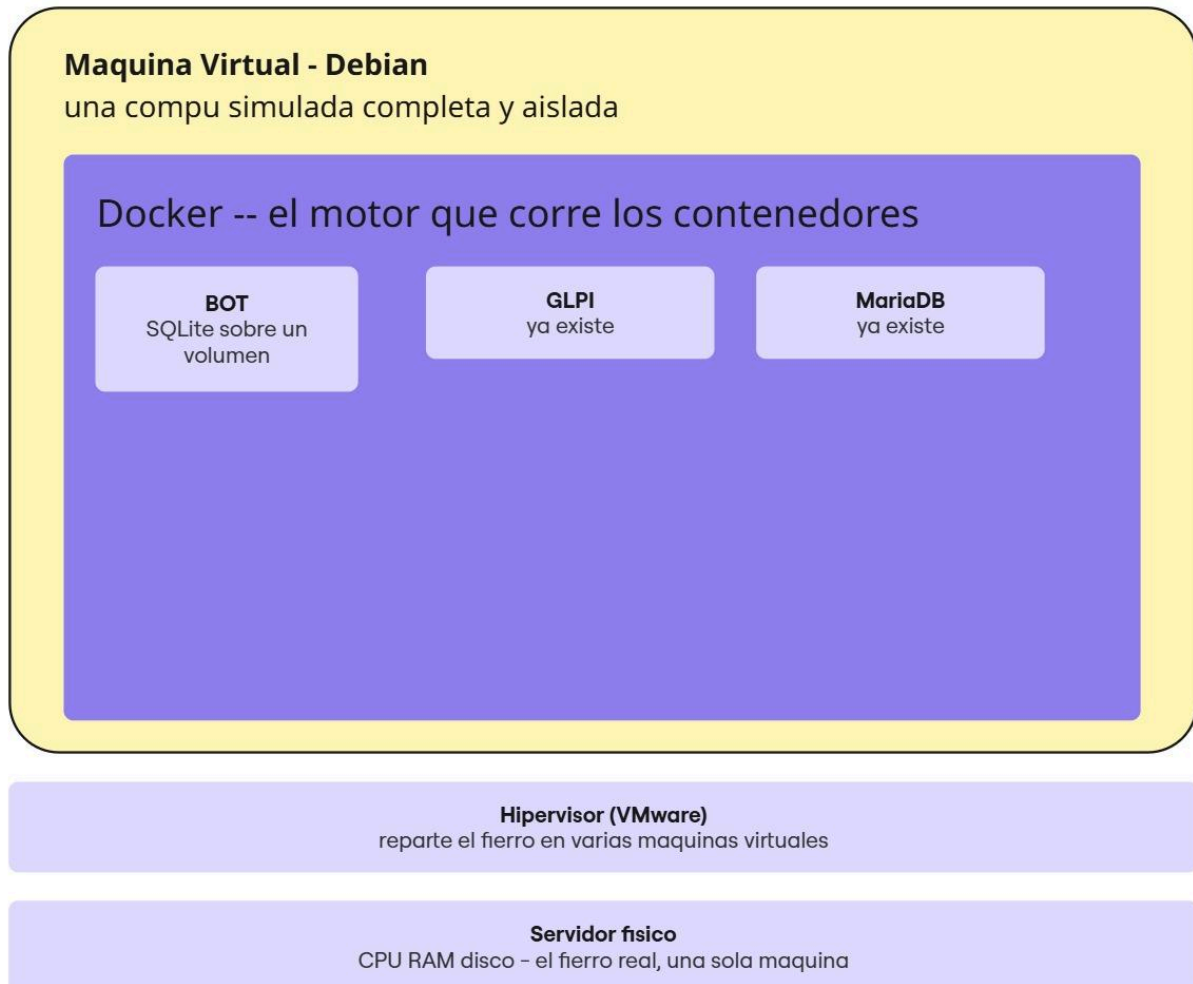
Las piezas son:

- **El bot** (Fast API + Python): el núcleo del proyecto. Internamente tiene un receptor de webhook (recibe los mensajes), un motor de conversación (la lógica) y el envío de mensajes (las respuestas).
- **GLPI** (API REST): el sistema de tickets ya existente. El bot se comunica con él exclusivamente por su API REST, nunca contra su base de datos. Es donde se crean los tickets, se consultan los tipos y categorías, y se deriva al área de soporte. GLPI tiene por detrás su propia base interna MariaDB, que el bot no toca.
- **Base del bot** (SQLite sobre un volumen): almacena el estado de las conversaciones, usando el número de WhatsApp como identificador. No es un contenedor aparte: es un archivo embebido en el contenedor del bot, alojado en un volumen para que sobreviva a los reinicios.
- **Servicios externos**: WhatsApp (Meta Cloud API) como canal de entrada y salida de mensajes; Gemini (Google AI) como motor de clasificación; y el usuario empleado, que interactúa a través de WhatsApp.

2. ¿Dónde vive el bot? Arquitectura por capas

La infraestructura se entiende mejor como una torre de capas, donde cada capa se apoya sobre la de abajo. Cuanto más abajo, más cerca del hardware real; cuanto más arriba, más virtual.

[Figura 2: Estructura por capas — dónde vive el bot]



- **Servidor físico:** una computadora real de la empresa (CPU, RAM, discos), pensada para estar encendida permanentemente y correr servicios. Si se instalara todo directamente sobre él, quedaría todo mezclado y un conflicto podría afectar a los demás servicios.
- **Hipervisor (VMware):** software que divide el servidor físico en varias computadoras virtuales independientes, repartiendo los recursos reales entre ellas. Desde afuera sigue

siendo un solo servidor, pero por dentro funciona como si hubiera varias máquinas separadas.

- **Máquina virtual (Debian):** una computadora completa simulada, con su propio sistema operativo y su espacio de trabajo aislado. Es la VM asignada para el proyecto, donde se puede experimentar sin afectar a los sistemas productivos.
- **Docker:** el motor que crea y administra los contenedores sobre la VM. Un contenedor, a diferencia de una máquina virtual, no simula una computadora completa: comparte el sistema operativo de la VM e incluye solo la aplicación y sus dependencias mínimas. Son "islas" livianas y aisladas.
- **Contenedores:** GLPI vive en un contenedor, MariaDB en otro, y el bot vivirá en un tercero. Los tres corren sobre la misma VM Debian pero aislados entre sí: si el bot falla, GLPI y MariaDB siguen funcionando.

Dos conceptos asociados: una **imagen Docker** es la plantilla o molde que contiene las instrucciones para crear el contenedor (Python, las librerías y el código del bot) — la relación imagen/contenedor es como la de un plano y una casa. Y **Portainer** es una interfaz web sobre Docker que permite ver, reiniciar y administrar los contenedores sin escribir tantos comandos.

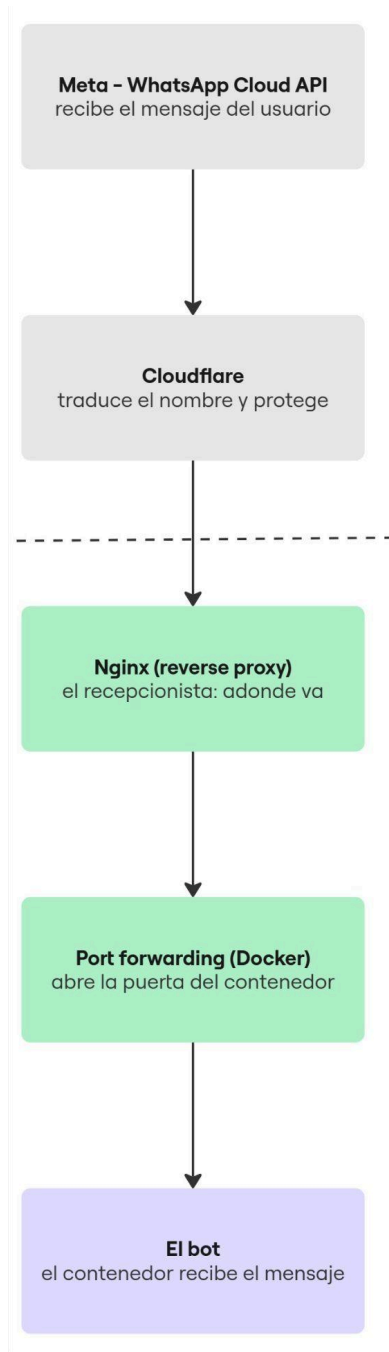
La imagen mental resultante es:

Servidor físico → Hipervisor (VMware) → VM Debian → Docker → Contenedor del bot

3. El camino de un mensaje

Esta vista responde una pregunta concreta: ¿cómo hace un mensaje enviado desde el celular de un empleado para atravesar todas las capas y llegar al contenedor del bot? La forma más fácil de entenderlo es pensar que el mensaje atraviesa varios "porteros", cada uno con una función específica, que lo van dejando pasar al siguiente.

[Figura 3: El camino de los mensajes]



El recorrido de entrada es:

Empleado → Meta (WhatsApp) → Cloudflare → Nginx → Docker / Port Forwarding
→ Bot → GLPI (si crea un ticket)

- **Primer portero — Meta:** el empleado escribe por WhatsApp; el mensaje llega primero a los servidores de Meta, que actúan como un cartero. Para entregarlo, Meta necesita la dirección pública del bot (por ejemplo, <https://bot.accionlist.com/webhook>).

Cada mensaje se convierte en una solicitud HTTP a esa dirección: eso es el webhook, la puerta donde Meta golpea para avisar que llegó un mensaje nuevo.

- **Segundo portero — Cloudflare:** Internet trabaja con direcciones IP, no con nombres. Cloudflare funciona como una guía telefónica que traduce el nombre (bot.accionlist.com) a la IP real del servidor. Además agrega seguridad: oculta la IP real de la empresa y filtra tráfico malicioso antes de que llegue a la infraestructura interna. Es un escudo delante del servidor.
- **Frontera internet / empresa:** hasta Cloudflare el mensaje viaja por internet. Una vez que se sabe a qué servidor enviarlo, cruza hacia la infraestructura interna de la empresa.
- **Tercer portero — Nginx:** cuando el mensaje llega al servidor, no va directo al bot. Primero lo recibe Nginx, que funciona como el recepcionista principal del edificio. Varios servicios (glpi.accionlist.com, bot.accionlist.com, intranet.accionlist.com) entran por la misma puerta; Nginx mira la dirección solicitada y decide a quién entregar cada pedido. Así los servicios internos quedan ocultos y organizados.
- **Cuarto portero — Docker y los puertos:** el contenedor no expone directamente sus puertos internos. Un puerto es un número que identifica qué programa debe recibir una conexión. Docker hace un mapeo (por ejemplo, puerto externo 17001 → puerto interno 80): cuando alguien llega al puerto externo, Docker redirige la conexión al puerto interno del contenedor. Eso es el port forwarding: el puerto externo es el que ve el mundo, el interno es el que usa realmente la aplicación.
- **Destino final — el bot:** tras atravesar Meta, Cloudflare, Nginx y Docker, el mensaje llega al contenedor donde corre la aplicación. Ahí ocurre la lógica real: leer el mensaje, clasificar el problema, consultar información, crear el ticket en GLPI y generar la respuesta.

El **camino de vuelta** es mucho más simple. El bot no necesita que nadie entre: simplemente hace una llamada saliente a la API de Meta, que entrega la respuesta al WhatsApp del empleado. Por eso suele decirse que "salir es fácil, entrar es difícil": la mayor parte de la infraestructura existe para permitir que una solicitud externa encuentre correctamente la aplicación.

Responsabilidad del proyecto

Toda esta infraestructura compleja ya existe en la empresa: no hay que instalar Cloudflare, construir Nginx ni diseñar la red desde cero. La responsabilidad del proyecto se reduce a cinco puntos:

1. Crear el bot.
2. Empaquetarlo en un contenedor Docker.
3. Hacer que escuche en un puerto.
4. Tener una URL pública asociada al webhook.
5. Registrar esa URL en Meta.

El resto del recorrido ya está montado por la infraestructura de la empresa.

Diseño BOT

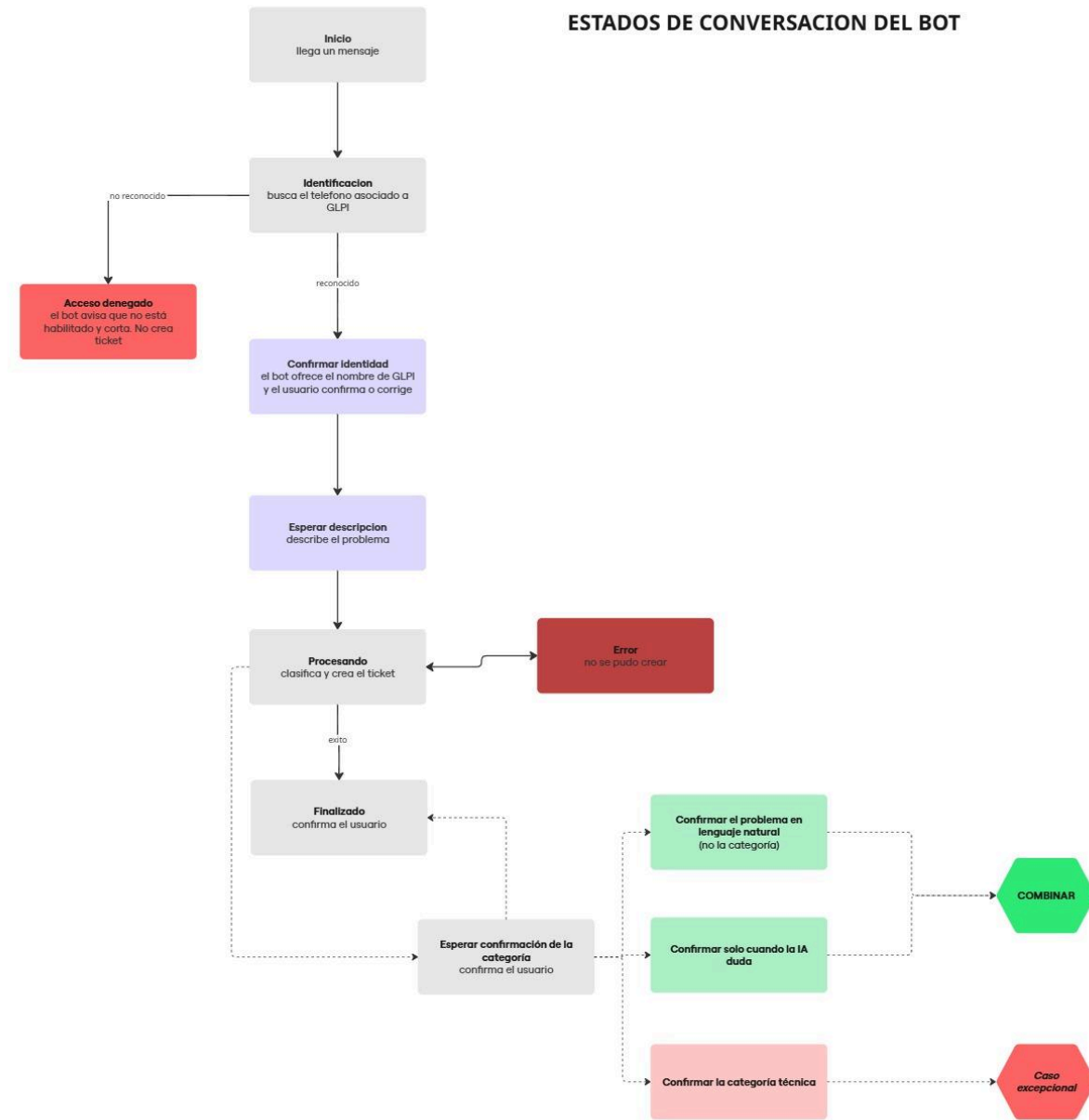
Introducción

Esta sección describe el diseño del bot a partir de los requerimientos y la arquitectura definidos en la fase de análisis. Se compone de cuatro piezas que se complementan: la máquina de estados de la conversación (cómo se comporta el bot turno a turno), el modelo de datos de persistencia (qué recuerda entre mensajes), la arquitectura interna por módulos (cómo está organizado por dentro) y el diagrama de secuencia (cómo viajan los mensajes entre los módulos a lo largo del tiempo).

1. Máquina de estados de la conversación

El bot sostiene conversaciones de varios turnos (multi-turno). Como el webhook de WhatsApp es *stateless* —cada mensaje llega como una petición independiente y aislada—, el bot necesita recordar en qué punto del flujo quedó cada conversación. La máquina de estados define esos "momentos" y cómo se pasa de uno a otro según lo que hace el usuario.

[Figura 1: Estados de conversación del bot]



Estados

Estado	Tipo	Qué pasa
Inicio	Interno	Llega un mensaje de un número sin conversación activa.
Identificación	Interno	El bot busca el teléfono en GLPI.
Pedir datos	Espera	No reconoció al usuario: le pide nombre, área y planta.

Esperar descripción	Espera	El bot espera que el usuario cuente el problema.
Procesando	Interno	Clasifica con IA, consulta categorías y crea el ticket en GLPI.
Error	Espera	Falló la creación; el usuario decide reintentar o cancelar.
Finalizado	Terminal	Ticket creado; el bot confirma con el número de ticket.

Transiciones

Desde	Evento	Hacia
Inicio	siempre	Identificación
Identificación	reconoce al usuario	Esperar descripción
Identificación	no lo reconoce	Pedir datos
Pedir datos	usuario completa sus datos	Esperar descripción
Esperar descripción	la descripción es insuficiente	Esperar descripción (repregunta)
Esperar descripción	descripción suficiente	Procesando
Esperar confirmación	el usuario confirma	Finalizado
Esperar confirmación	el usuario corrige o desambigua	Esperar confirmación
Procesando	ticket creado OK	Finalizado
Procesando	falla GLPI	Error
Error	el usuario reintenta	Procesando
Error	el usuario cancela	Inicio / Finalizado

Consideraciones de diseño

Solo los estados de espera (Pedir datos, Esperar descripción y Error) se persisten en la base de datos: son los puntos donde el bot "duerme" entre un mensaje y el siguiente, así que necesita recordar en cuál quedó cada conversación. Los estados internos (Inicio, Identificación, Procesando) ocurren todos dentro de un mismo turno, por lo que no se almacenan como estado.

2. Modelo de datos (persistencia en SQLite)

El estado de las conversaciones se almacena en una base SQLite embebida, alojada en un volumen persistente del contenedor del bot. SQLite no es un servidor de base de datos aparte (a diferencia de la MariaDB de GLPI): es un único archivo en disco que el propio bot lee y escribe, sin sumar otro contenedor. Al estar en un volumen, el archivo sobrevive a los reinicios y actualizaciones del contenedor.

[Figura 2: Modelo de datos — tabla de conversaciones en SQLite]

	Estado	Tipo	Que pasa
1	Inicio	Interno	Llega un mensaje de un numero sin conversacion...
2	Identificacion	Interno	El bot busca el telefono en GLPI
3	Pedir datos	Espera	No reconocio al usuario: le pide nombre, area y...
4	Esperar descripcion	Espera	El bot espera que el usuario cuente el problema
5	Esperar confirmación	Espera	El bot espera que el usuario confirme la categoria
6	Procesando	Interno	Clasifica con IA, consulta categorias y crea el ticket e...
7	Error	Espera	Falló la creacion; el usuario decide reintentar o...
8	Finalizado	Terminal	Ticket creado; el bot confirma con el numero

```
CREATE TABLE conversaciones (  
  telefono      TEXT PRIMARY KEY,      -- número de WhatsApp, identifica la conversación  
  estado       TEXT NOT NULL,         -- estado actual ('identificacion', 'pidiendo_datos',  
  'esperando_descripcion', 'error'...)  
  identificado  INTEGER NOT NULL DEFAULT 0, -- 0 = no reconocido en GLPI, 1 = sí  
  usuario_nombre TEXT,                -- traído de GLPI o ingresado a mano  
  usuario_area  TEXT,  
  usuario_planta TEXT,  
  descripcion   TEXT,                -- problema acumulado (puede venir en varios mensajes)  
  categoria_glpi TEXT,                -- categoría clasificada (se llena en Procesando)  
  ticket_id     TEXT,                -- id del ticket en GLPI (se llena al pasar a Finalizado)  
  creado_en     TEXT NOT NULL DEFAULT (datetime('now')),  
  actualizado_en TEXT NOT NULL DEFAULT (datetime('now'))  
);
```

Cómo se llena, siguiendo la máquina de estados

- telefono, estado y los timestamps están desde el primer mensaje.
- identificado y los usuario_* se completan en Identificación o Pedir datos.
- descripcion se va acumulando en Esperar descripción.
- categoria_glpi se llena cuando clasifica (Procesando), y
- ticket_id cuando crea el ticket (Finalizado).

Guardar la categoría antes de crear el ticket te da un bonus: si falla y cae en Error, al reintentar no necesitás reclasificar, ya la tenés.

Decisiones de diseño

Teléfono como clave primaria. Implica una conversación activa por número a la vez, lo más simple y consistente con que el historial de incidentes reside en GLPI y no se duplica aquí. Si en el futuro se quisiera conservar conversaciones cerradas, se migraría a un identificador autoincremental con el teléfono indexado.

Timestamps como TEXT. SQLite no tiene un tipo de dato dedicado para fechas. La función `datetime('now')` devuelve un texto en formato ISO, ordenable y legible al inspeccionar la base. La alternativa sería un INTEGER con *epoch* Unix, más compacto pero menos legible.

identificado como INTEGER (0/1). SQLite no tiene tipo booleano; se representa con 0 y 1.

estado como texto libre. La validez de los valores se controla en el código (un *enum* con los nombres de los estados). Opcionalmente puede agregarse una restricción **CHECK** en la base.

actualizado_en. Permite limpiar conversaciones abandonadas (usuarios que iniciaron y nunca respondieron) mediante una tarea que borre filas sin actividad reciente, evitando que la base acumule sesiones muertas.

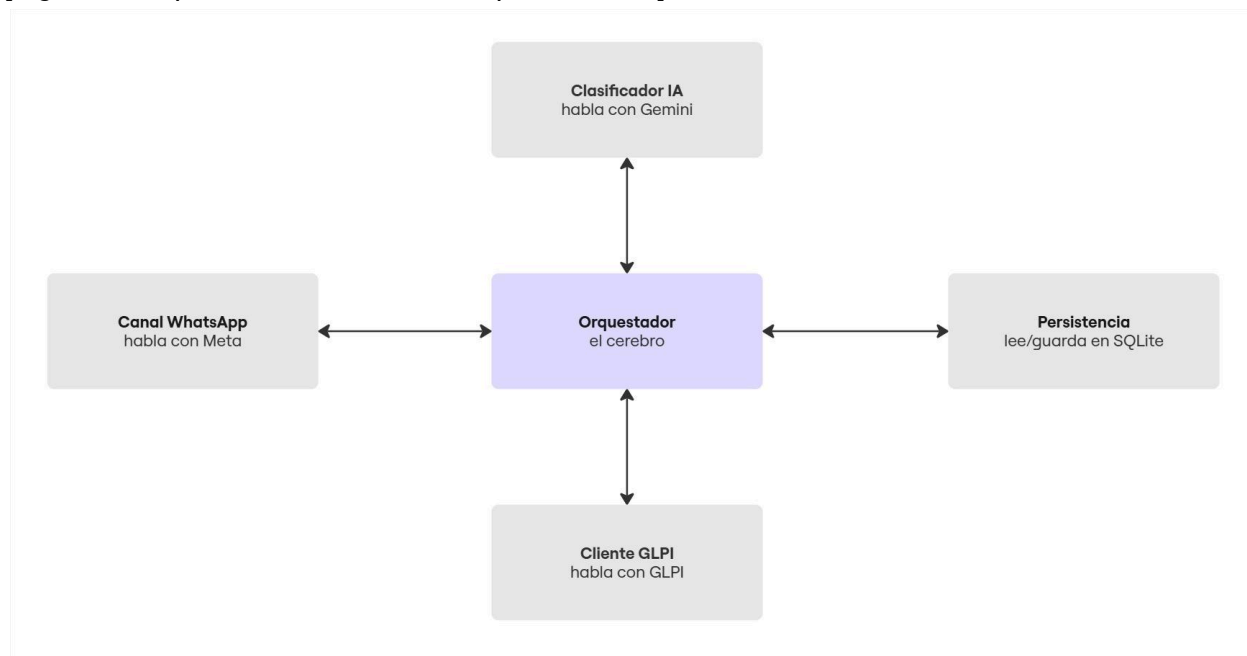
Nota sobre la identificación del usuario

La identificación no se realiza contra Active Directory / LDAP, sino contra GLPI: el bot consulta a GLPI si existe un usuario registrado con el número de teléfono que escribe. Si existe, GLPI devuelve los datos de la persona y el bot la identifica; si no, el bot solicita los datos por chat. Esto evita sumar una integración adicional, ya que la comunicación con GLPI es necesaria de todos modos.

3. Arquitectura interna por módulos

El bot se organiza internamente en módulos, cada uno con una responsabilidad única. Esta separación es la que permite cumplir el requerimiento de modularidad y reutilización: poder extender el bot a otras áreas (o canales) sin reescribir su núcleo.

[Figura 3: Arquitectura interna del bot por módulos]



Los módulos son:

1. **Canal WhatsApp** — recibe los mensajes que entran por el webhook de Meta y envía las respuestas de vuelta. Es la puerta de entrada y salida.
2. **Orquestador** — contiene la máquina de estados y decide qué hacer en cada momento. Coordina a los demás módulos.
3. **Persistencia** — guarda y lee el estado de cada conversación en el SQLite.
4. **Cliente GLPI** — concentra toda la comunicación con GLPI: identificar al usuario por su número, consultar categorías y crear el ticket.
5. **Clasificador IA** — envía el problema a Gemini y obtiene la categoría.

El **Orquestador** es el centro de la arquitectura. Es el único que conoce la máquina de estados y el único que coordina: le solicita el estado a Persistencia, la categoría al Clasificador, la creación del ticket al Cliente GLPI, etc. Los demás módulos no se comunican entre sí —todo el tráfico pasa por el Orquestador—, lo que mantiene cada módulo independiente y reemplazable.

Además, el Orquestador no toca ningún sistema externo: no sabe de Meta, ni de GLPI, ni de Gemini. Solo conoce la máquina de estados y delega en sus módulos. Por eso, sumar un nuevo canal (por ejemplo, Telegram) implicaría agregar un módulo de canal sin modificar el Orquestador.

Cada módulo se corresponde con un archivo del proyecto, lo que traslada esta arquitectura directamente a la estructura de carpetas del código:

bot/

├── main.py # arranca FastAPI y recibe el webhook

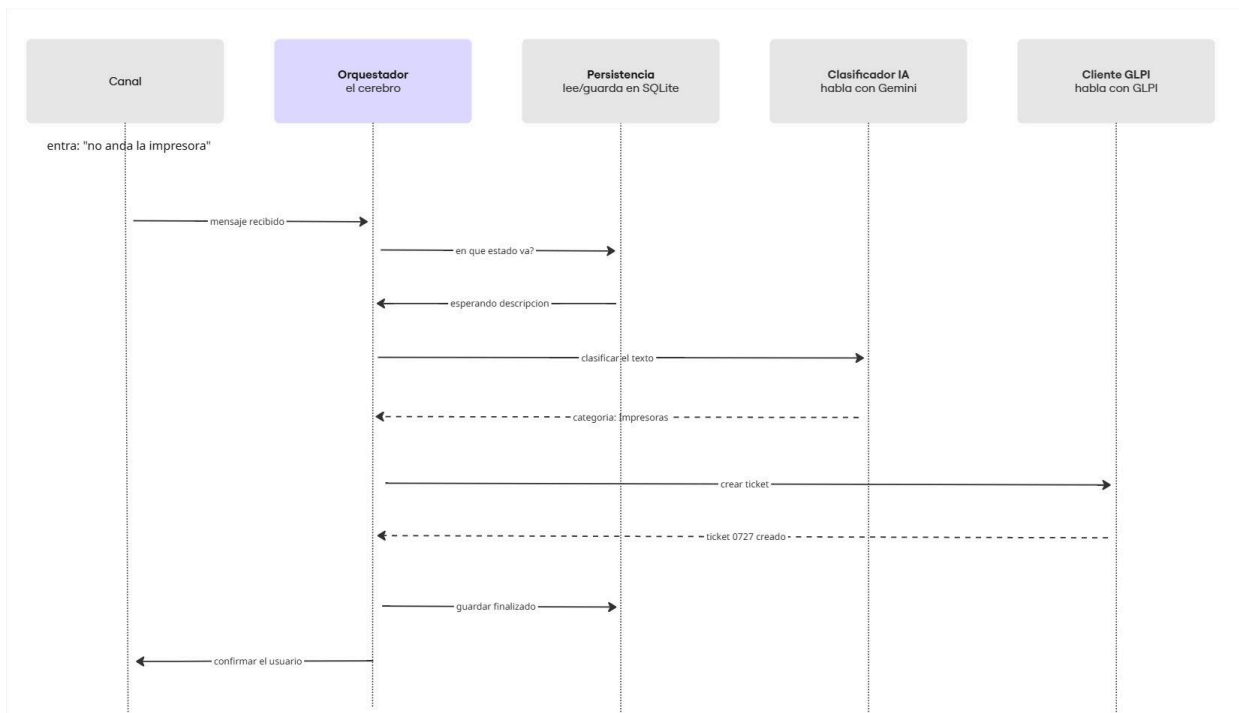
- canal_whatsapp.py # módulo Canal WhatsApp
- orquestador.py # módulo Orquestador (la máquina de estados)
- persistencia.py # módulo Persistencia (SQLite)
- cliente_glpi.py # módulo Cliente GLPI
- clasificador.py # módulo Clasificador IA

4. Diagrama de secuencia

Mientras el diagrama de arquitectura muestra qué módulo se conecta con cuál (una vista estática), el diagrama de secuencia muestra el orden temporal: cómo los módulos se mandan mensajes a lo largo del tiempo. Se lee de arriba hacia abajo. Las flechas llenas representan pedidos y las punteadas, las respuestas.

El siguiente diagrama detalla el segundo turno de una conversación (el usuario ya fue identificado y ahora describe su problema, el bot lo clasifica y crea el ticket):

[Figura 4: Diagrama de secuencia — segundo turno (clasificar y crear el ticket)]



Lo primero que hace el Orquestador al recibir el mensaje es consultarle a Persistencia en qué estado había quedado la conversación; la respuesta ("esperando descripción") le indica que el usuario ya estaba identificado y que lo que faltaba era el detalle del problema. Este paso es el bot "despertándose": retoma exactamente donde había quedado, gracias al estado almacenado. A partir de ahí, el Orquestador pide la clasificación al Clasificador IA, ordena al

Cliente GLPI crear el ticket, guarda el estado final en Persistencia y, por último, confirma al usuario a través del Canal.

El primer turno (recepción del mensaje, identificación contra GLPI y solicitud de la descripción) sigue el mismo patrón de coordinación a través del Orquestador.
